

Verification and Validation (V&V)



Verification and Validation(V&V)

- Software V&V is a process implemented throughout the software life cycle to ensure the following:
- Verification: Each step in the development process is carried out correctly
 - Are we building the product right?
 - mostly inspections and reviews
- Validation: Each artifact produced meets its requirements as specified in prior phases.
 - Are we building the right product?
 - mostly testing



Inspections

- Analysis of the static system representation to discover problems (static verification)
- Does NOT require execution of a system
 - It can be used before implementation.
- May apply to any representation of the system
 - requirements, design, configuration data, test data, etc.
- “Best Known Method” (BKM) for increasing quality
 - Developed by Michael Fagan at IBM, paper published 1976
 - Inspections of design and code usually remove 50-90% of defects before testing
 - Very economical compared to testing

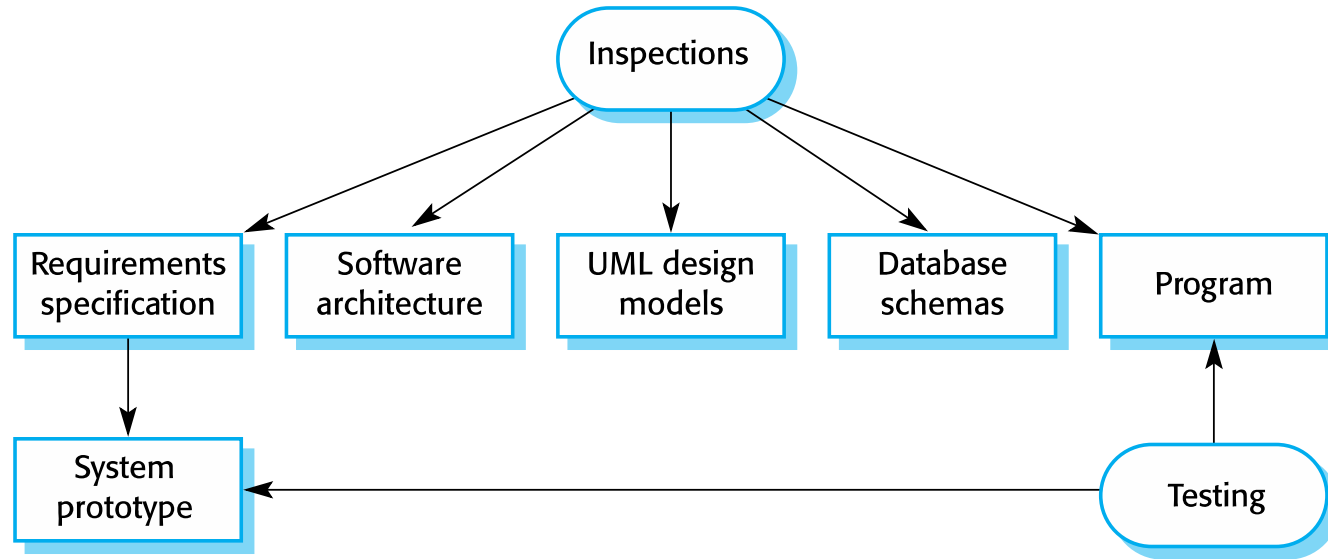


Program testing

- Testing
 - To show that a program does what it is intended to do
 - To discover program defects before it is put into use
 - Can reveal the presence of errors NOT their absence
- Part of Verification and validation
 - Dynamic technique for verification/validation process



Inspections and testing



- Inspections and testing are complementary
- Both should be used during the V & V process
- Inspections
 - cannot check conformance with the customer's real requirements.
 - cannot check non-functional characteristics such as performance, usability, etc.

Stages of testing

- Development testing
 - the system is tested during development to discover bugs and defects.
- Release testing
 - a separate testing team test a complete version of the system before it is released to users.
- User testing
 - users or potential users of a system test the system in their own environment.



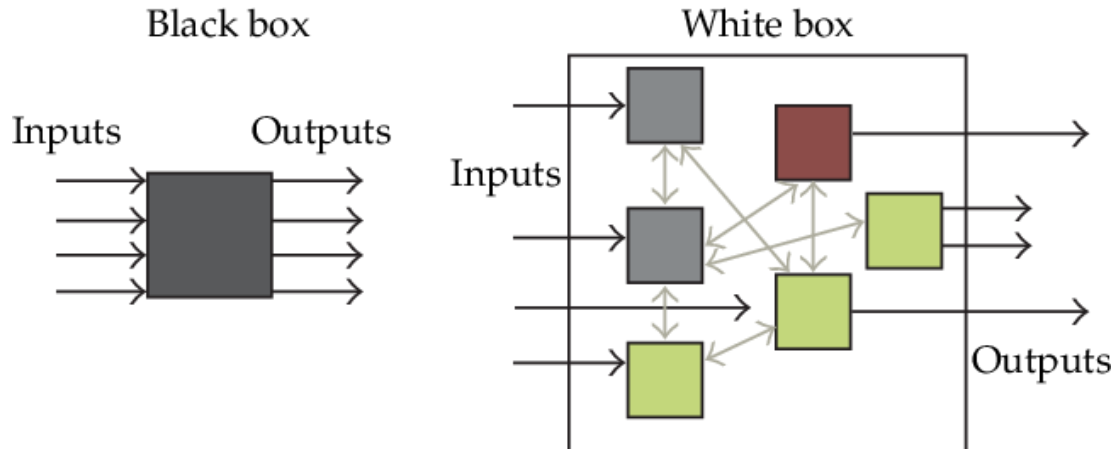
Development testing

- Unit testing
 - individual program units or object classes are tested
 - Unit testing should focus on testing the functionality of objects or methods.
- Component testing
 - several individual units are integrated to create composite components.
 - component testing should focus on testing component interfaces.
- System testing
 - some or all of the components in a system are integrated and the system is tested as a whole.
 - system testing should focus on testing component interactions.



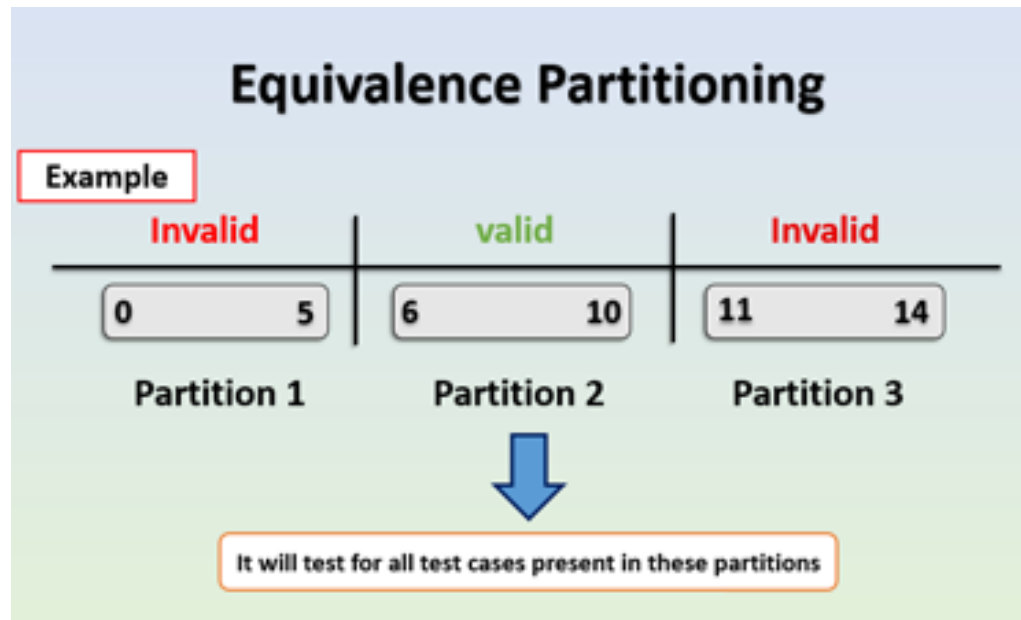
Black box testing V.S. White box (Structural) testing

- Black-box testing
 - Takes an external perspective of the test object to derive test cases
 - The test designer selects valid and invalid input and determines the correct output
 - There is no knowledge of the test object's internal structure
- White-box testing
 - Derivation of test cases according to program structure.
 - Knowledge of the program is used to identify additional test cases



Equivalence Partition

- Input data and output results often fall into different classes where all members of a class are related.
- Each of these classes is an equivalence partition or domain where the program behaves in an equivalent way for each class member.
- Test cases should be chosen from each partition.



Search routine specification

(Note that we **do not know** the implementation details)

```
procedure Search (Key : ELEM ; T: SEQ of ELEM;  
    Found : in out BOOLEAN; L: in out ELEM_INDEX) ;
```

Pre-condition

```
-- the sequence has at least one element  
T'FIRST <= T'LAST
```

Post-condition

```
-- the element is found and is referenced by L  
( Found and T (L) = Key)
```

or

```
-- the element is not in the array  
( not Found and  
not (exists i, T'FIRST >= i <= T'LAST, T (i) = Key ))
```



Search routine - input partitions (Black Box Test)

- Two equivalence partitions
 - Inputs where the key element is a member of the array.
 - Inputs where the key element is not a member of the array.
- Two more equivalence partitions
 - The input sequence has a single value
 - The number of elements in the sequence > 1



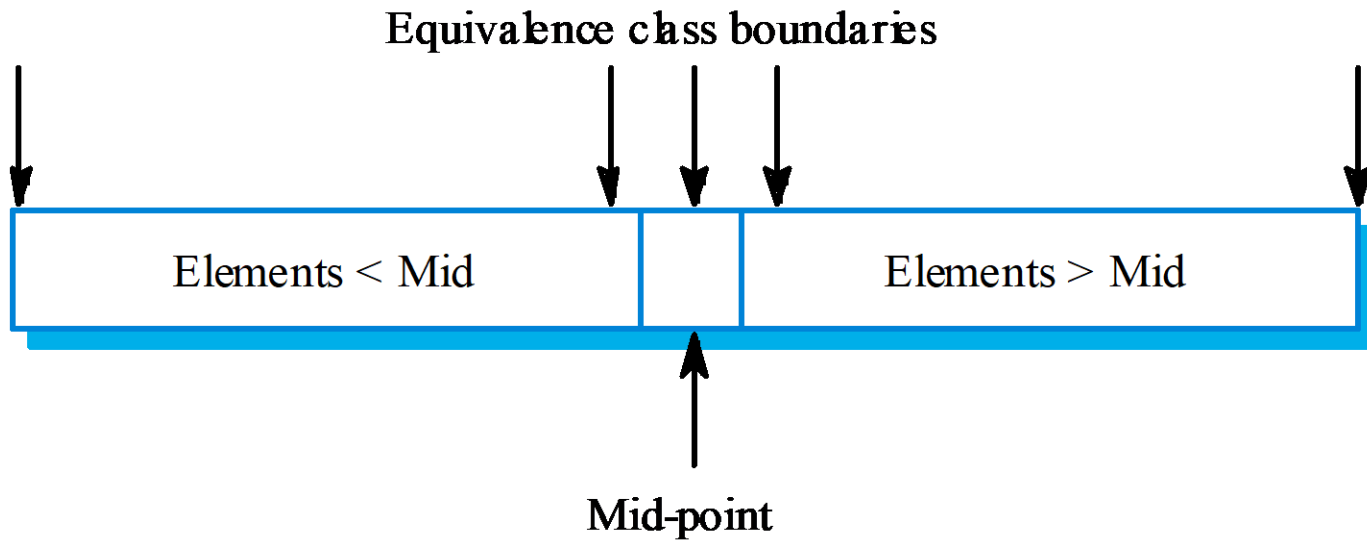
Search routine - input partitions (Black Box Test)

Sequence	Element
Single value	In sequence
Single value	Not in sequence
More than 1 value	First element in sequence
More than 1 value	Last element in sequence
More than 1 value	Middle element in sequence
More than 1 value	Not in sequence

Input sequence (T)	Key (Key)	Output (Found, L)
17	17	true, 1
17	0	false, ??
17, 29, 21, 23	17	true, 1
41, 18, 9, 31, 30, 16, 45	45	true, 7
17, 18, 21, 23, 29, 41, 38	23	true, 4
21, 23, 29, 33, 38	25	false, ??



Binary search equiv. partitions (White Box Test)



Binary search - test cases

(White Box Test)

Input array (T)	Key (Key)	Output (Found, L)
17	17	true, 1
17	0	false, ??
17, 21, 23, 29	17	true, 1
9, 16, 18, 30, 31, 41, 45	45	true, 7
17, 18, 21, 23, 29, 38, 41	23	true, 4
17, 18, 21, 23, 29, 33, 38	21	true, 3
12, 18, 21, 23, 32	23	true, 4
21, 23, 29, 33, 38	25	false, ??



Testing Stages

- White-box testing:
 - performed by developers
 - Development testing
- Black-box testing:
 - performed by separate testing team
 - Release testing
 - User testing